PROJECT:

RAG using Langchain

```
from google.colab import userdata
x = userdata.get('GROQ_API_KEY')
```

```
groq_api_key=x
```

```
# !pip install langchain-community langchain-openai langchain -q

# !pip install --upgrade langchain-community langchain-openai langchain -q

# !pip install langchain-classic langchain-openai -q
# !pip install pypdf -q
# !pip install langchain-google-genai -q
# !pip install langchain-community sentence-transformers -q
# !pip install langchain-groq -q
```

```
import os
from langchain_community.document_loaders import PyPDFLoader, PyMuPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain_classic.chains import RetrievalQA
from langchain_groq import ChatGroq
from pathlib import Path
```

Start coding or generate with AI.

```
### Read all the pdf's inside the directory
def process_all_pdfs(pdf_directory):
    """Process all PDF files in a directory"""
    all_documents = []
    pdf_dir = Path(pdf_directory)

    # Find all PDF files recursively
    pdf_files = list(pdf_dir.glob("**/*.pdf"))

    print(f"Found {len(pdf_files)} PDF files to process")

    for pdf_file in pdf_files:
        print(f"\nProcessing: {pdf_file.name}")
        try:
            loader = PyPDFLoader(str(pdf_file))
            documents = loader.load()

            # Add source information to metadata
            for doc in documents:
                doc.metadata['source_file'] = pdf_file.name
                doc.metadata['file_type'] = 'pdf'

            all_documents.extend(documents)
            print(f"  ✓ Loaded {len(documents)} pages")

        except Exception as e:
            print(f"  ✗ Error: {e}")

    print(f"\nTotal documents loaded: {len(all_documents)}")
    return all_documents

# Process all PDFs in the data directory
all_pdf_documents = process_all_pdfs("/content/data")
```

```
Found 4 PDF files to process

Processing: Prompt Engineering.pdf
  ✓ Loaded 24 pages
```

```
Processing: chain of thoughts prompting.pdf
  ✓ Loaded 43 pages

Processing: RNN research paper.pdf
  ✓ Loaded 43 pages

Processing: colah's blog.pdf
  ✓ Loaded 9 pages

Total documents loaded: 119
```

```
all_pdf_documents
```

```
[Document(metadata={'producer': 'Skia/PDF m144 Google Docs Renderer', 'creator': 'PyPDF', 'creationdate': '', 'title': 'Prompt
Engineering', 'source': '/content/data/Prompt Engineering.pdf', 'total_pages': 24, 'page': 0, 'page_label': '1',
'source_file': 'Prompt Engineering.pdf', 'file_type': 'pdf'}, page_content='What is a Prompt? A prompt is the input
or instruction you give to a large language model (LLM) like \nChatGPT,\n \nGPT-4,\n \nClaude,\n \nMistral,\n
\netc.,\n \nto\n \nget\n \na\n \ndesired\n \noutput.\n Prompt = Instruction that guides the model’s behavior \nPrompt
engineering is the art of talking to an AI (like me) in a smart way — \ncrafting\n \nyour\n \nrequest\n
\n(prompt)\n \nso\n \nthat\n \nthe\n \nAI\n \ngives\n \nyou\n \nthe\n \nmost\n \nuseful,\n \naccurate,\n \nor\n \ncreative\n
\nanswer.\n \n \nIt’s not about what you ask, but how you ask it. \n \nThink of it as learning how to “speak
Genie language” — the clearer and more \ndetailed\n \nyour\n \nwish,\n \nthe\n \nbetter\n \nthe\n \nmagic\n \nworks!\n
\n \nWithout prompt engineering: \nYou: “Genie, I want money.” \n \nGenie:\n \nPoof!\n \nYou\n \nget\n \n$1\n \n—\n
\ntechnically\n \nmoney.\n \n \nWith prompt engineering: \nYou: “Genie, please give me $1 million in my bank
account, legally \nearned,\n \nin\n \nmy\n \nname,\n \nwithout\n \nharming\n \nanyone.”\n \n \nGenie:\n \nPoof!\n \nNow\n
\nyou\n \nget\n \nexactly\n \nwhat\n \nyou\n \nmeant,\n \nnot\n \njust\n \nwhat\n \nyou\n \nsaid.\n \n \nLesson: A vague
prompt gives vague results. A well-engineered prompt \ngives\n \nprecise,\n \nmeaningful\n \nresults.'),
 Document(metadata={'producer': 'Skia/PDF m144 Google Docs Renderer', 'creator': 'PyPDF', 'creationdate': '', 'title': 'Prompt
Engineering', 'source': '/content/data/Prompt Engineering.pdf', 'total_pages': 24, 'page': 1, 'page_label': '2',
'source_file': 'Prompt Engineering.pdf', 'file_type': 'pdf'}, page_content='Without prompt engineering: \nWaiter: “What
would you like?” \n \nYou:\n \n“Food.”\n \n \nWaiter:\n \n“Okay…”\n \nbrings\n \nyou\n \nplain\n \ntoast.\n \n \nWith
prompt engineering: \nYou: “I’d like a medium-rare cheeseburger with extra pickles, no \nonions,\n \nand\n \na\n
\nside\n \nof\n \ncrispy\n \nfries.”\n \n \nWaiter:\n \n“Got\n \nit!”\n \n—\n \nyou\n \nget\n \nexactly\n \nwhat\n \nyou\n
\nwanted.\n \n \nLesson: When you describe your order (prompt) clearly, you get a better “output” \n(answer).\n
\n \nWhy It’s Important \nPrompt engineering helps you: \n>Guide the AI toward your goal (creative writing,
analysis, summaries, etc.) \n>Save\n \ntime\n \nby\n \nreducing\n \nback-and-forth\n \nclarifications.\n \n>Get\n \nmore\n
\nrelevant\n,\n \naccurate\n,\n \nand\n \ncontext-aware\n \nresults.\n \n \nPoor prompt → Wasted time, generic output,
hallucinations Great prompt → Useful summaries, structured insights, accurate automation \n \nWhat is a Good
Prompt? A good prompt is clear, specific, structured, and goal-driven. Prompt:'),
 Document(metadata={'producer': 'Skia/PDF m144 Google Docs Renderer', 'creator': 'PyPDF', 'creationdate': '', 'title': 'Prompt
Engineering', 'source': '/content/data/Prompt Engineering.pdf', 'total_pages': 24, 'page': 2, 'page_label': '3',
'source_file': 'Prompt Engineering.pdf', 'file_type': 'pdf'}, page_content='You are a senior technical writer at
Oracle. Write a 100-word summary explaining \nhow\n \nrelational\n \ndatabases\n \nwork,\n \nintended\n \nfor\n \na\n
\nnew\n \nhire\n \nwith\n \nno\n \ndatabase\n \nbackground.\n Why is the above is good prompt? >Defines role (senior
writer) >Defines audience (new hire, non-technical) >Sets length constraint (100 words) >Specifies content type
(relational databases) Prompt 1 (Bad): Explain OCI. Prompt 2 (Good): You are a cloud solutions architect at
Oracle. Explain Oracle Cloud Infrastructure \n(OCI)\n \nto\n \na\n \ngroup\n \nof\n \nfinance\n \nexecutives\n \nwho\n
\nare\n \nconsidering\n \nmigrating\n \nworkloads\n \nfrom\n \non-premise\n \nsystems.\n \nUse\n \nsimple\n \nlanguage\n
\nand\n \nlimit\n \nto\n \n200\n \nwords.\n Step-by-Step Prompt Evolution \nBad Prompt (Initial) Write about time
management. Step 1: Add Role You are a productivity coach. Write about time management. \nStep 2: Add
Audience You are a productivity coach. Write about time management for college students.'),
 Document(metadata={'producer': 'Skia/PDF m144 Google Docs Renderer', 'creator': 'PyPDF', 'creationdate': '', 'title': 'Prompt
Engineering', 'source': '/content/data/Prompt Engineering.pdf', 'total_pages': 24, 'page': 3, 'page_label': '4',
'source_file': 'Prompt Engineering.pdf', 'file_type': 'pdf'}, page_content='Step 3: Add Task Focus You are a
productivity coach. Write 3 time management tips for college students. \nStep 4: Add Constraints and Format
You are a productivity coach. Write exactly 3 time management tips for college students. Use bullet points.
Keep each tip under 25 words. \nFinal Optimized Prompt You are a productivity coach. Write exactly 3 time
management tips for college students. Use bullet points. Keep each tip under 25 words. Avoid technical jargon.
Make the tone casual and encouraging. Prompt Structure Role, Audience, Task, Constraints, Format, Tone \n
\nStep-by-Step Prompt Evolution (Oracle-Specific Example) \nBad Prompt (Initial) Explain Oracle Fusion ERP.'),
 Document(metadata={'producer': 'Skia/PDF m144 Google Docs Renderer', 'creator': 'PyPDF', 'creationdate': '', 'title': 'Prompt
Engineering', 'source': '/content/data/Prompt Engineering.pdf', 'total_pages': 24, 'page': 4, 'page_label': '5',
'source_file': 'Prompt Engineering.pdf', 'file_type': 'pdf'}, page_content='Step 1: Add Role You are a senior Oracle
consultant. Explain Oracle Fusion ERP. \nStep 2: Add Audience You are a senior Oracle consultant. Explain
Oracle Fusion ERP to a newly hired business analyst from a \nnon-technical\n \nbackground.\n \nStep 3: Add
Task Focus You are a senior Oracle consultant. Explain Oracle Fusion ERP to a newly hired business analyst
from a \nnon-technical\n \nbackground.\n Focus only on: what it is, its key modules, and how enterprises use
it. \nStep 4: Add Output Format and Constraints You are a senior Oracle consultant. Explain Oracle Fusion
ERP to a newly hired business analyst from a \nnon-technical\n \nbackground.\n Focus only on: what it is, its
key modules, and how enterprises use it. Return the answer in 5 bullet points. Each bullet must be under 25
```

```
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)
# Split the documents into smaller chunks
chunks = text_splitter.split_documents(all_pdf_documents) # This will return a list of chunks, each representing a smaller part

# Print the number of chunks created
```

```
print(f"Number of chunks created: {len(chunks)}") # This will print the number of chunks created from the documents
```

```
Number of chunks created: 804
```

```
print(chunks[2]) # Display the 3rd chunk (index 2) to see the content of the chunk
```

```
page_content='Genie:

Poof!

You

get

$1

–

technically

money.


With  prompt  engineering:
You:  “Genie,  please  give  me  $1  million  in  my  bank  account,  legally
earned,

in

my

name,

without

harming

anyone.”


Genie:

Poof!

Now

you

get

exactly

what

you

meant,

not

just

what
```

```
print(chunks[-3])
```

```
page_content='IEEE, 2017.
[73] P. Werbos. Backpropagation through time: what does it do and how to do it. In Proceedings of IEEE , volume 78, pages
1550–1560, 1990.
[74] P. J. Werbos. Generalization of backpropagation with application to a recurrent gas market model. Neural Networks, 1,
1988.
[75] Ronald J. Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks.
Neural Computation, 1:270–280, 1989.' metadata={'producer': 'pdfTeX-1.40.25', 'creator': 'LaTeX with hyperref', 'creationdate':
```

```
#from langchain_google_genai import GoogleGenerativeAIEmbeddings
```

```
#  embeddings = GoogleGenerativeAIEmbeddings(model="models/gemini-embedding-001", api_key=GEMINI_API_KEY)
```

```
#  vector = embeddings.embed_query("hello, world!")
#  vector[:5]
```

```
# if openAI api Key used
# embeddings = OpenAIEmbeddings(api_key=open_api_key)
```

```
# !pip install faiss-cpu -q
```

```
from langchain_community.embeddings import HuggingFaceEmbeddings

# Initialize the embeddings using a powerful, local model
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

# 2. Your vector store code (now corrected)
vectorstore = FAISS.from_documents(chunks, embeddings)
```

```
/tmp/ipython-input-3548034415.py:4: LangChainDeprecationWarning: The class `HuggingFaceEmbeddings` was deprecated in LangChain 0
  embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
modules.json: 100%  349/349 [00:00<00:00, 37.4kB/s]
config_sentence_transformers.json: 100%  116/116 [00:00<00:00, 13.3kB/s]
README.md: 10.5k/? [00:00<00:00, 925kB/s]
sentence_bert_config.json: 100%  53.0/53.0 [00:00<00:00, 6.42kB/s]
config.json: 100%  612/612 [00:00<00:00, 73.1kB/s]
model.safetensors: 100%  90.9M/90.9M [00:00<00:00, 151MB/s]
tokenizer_config.json: 100%  350/350 [00:00<00:00, 43.9kB/s]
vocab.txt: 232k/? [00:00<00:00, 9.81MB/s]
tokenizer.json: 466k/? [00:00<00:00, 25.0MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 13.5kB/s]
config.json: 100%  190/190 [00:00<00:00, 23.9kB/s]
```

```
vectorstore
```

```
<langchain_community.vectorstores.faiss.FAISS at 0x7a07105a8740>
```

```
docs = vectorstore.docstore._dict
print(len(docs))
print("."*70)
# docs
```

```
804
.....................................................................
```

```
# To see one random entry:

import random

random_id = random.choice(list(docs.keys()))
print("Random ID:", random_id)
print("Content:\n", docs[random_id].page_content)
print("Metadata:\n", docs[random_id].metadata)
```

```
Random ID: 68c56277-1c4b-4464-b026-833d521f1320
Content:
 Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea
Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, et al. 2022. Do as I can, not as I
say: Grounding language in robotic affordances. arXiv preprint arXiv:2204.01691.
Aida Amini, Saadia Gabriel, Shanchuan Lin, Rik Koncel-Kedziorski, Yejin Choi, and Hannaneh
Hajishirzi. 2019. MathQA: Towards interpretable math word problem solving with operation-
Metadata:
```

```
{'producer': 'pdfTeX-1.40.21', 'creator': 'LaTeX with hyperref', 'creationdate': '2023-01-12T01:06:30+00:00', 'author': '', 'ke
```

```
# Search by Query (Semantic Check)

# You can test whether your FAISS store is working semantically:
query = "LSTM"
results = vectorstore.similarity_search(query, k=2)
for i, r in enumerate(results):
    print(f"\nResult {i+1}:")
    print(r.page_content[:300], "...")
    print("Metadata:", r.metadata)


# This retrieves the top-2 most semantically similar chunks to your query.
# It’s a great debugging tool to see if your embeddings are meaningful.
```

```
Result 1:
together with detailed descriptions of its constituent entities. Albeit unconventional, our choice of notation and the method fo
presenting the LSTM system emphasizes ease of understanding. As part of the analysis, we identify new opportunities to enrich
the LSTM system and incorporate these extens ...
Metadata: {'producer': 'pdfTeX-1.40.25', 'creator': 'LaTeX with hyperref', 'creationdate': '2023-08-01T01:22:01+00:00', 'author'

Result 2:
explanation of all aspects of the Vanilla LSTM cell. Even though this section is intended to be self-contained, familiarity
with the material covered in the preceding sections will be beneficial. The Augmented LSTM system, which embellishes the
Vanilla LSTM system with the new computational componen ...
Metadata: {'producer': 'pdfTeX-1.40.25', 'creator': 'LaTeX with hyperref', 'creationdate': '2023-08-01T01:22:01+00:00', 'author'
```

```
# See Raw Embeddings (for Curiosity)

vector_dim = vectorstore.index.d
print("Vector dimension:", vector_dim)

# See first stored vector
print(vectorstore.index.reconstruct(0))

# You’ll see a list of 384 or 1536 floating-point numbers — these are high-dimensional coordinates representing the meaning of
# your text chunk.
# (Usually not human-readable, but useful for debugging or confirming embedding shape.)
```

```
Vector dimension: 384
[ 4.68287943e-03 -3.13701667e-02  2.31676623e-02 -3.08047719e-02
  2.42145676e-02 -5.38972057e-02  7.77483732e-02  7.35235661e-02
  2.59277746e-02 -3.44252400e-03 -4.70879301e-02 -3.50230262e-02
  6.62721172e-02 -6.31348118e-02  3.07386555e-02 -4.56063896e-02
  5.09584099e-02 -1.08680062e-01  5.01321666e-02 -2.20396873e-02
  2.33584065e-02  6.62494302e-02  6.21577501e-02  3.72870043e-02
 -2.58519147e-02  3.17570418e-02  4.97602038e-02 -1.45644033e-02
  4.11017872e-02  2.93317065e-02 -2.42912956e-02  3.56823877e-02
  1.20651141e-01  5.77866919e-02 -5.89059182e-02  1.44516248e-02
  1.47101991e-02 -4.50252779e-02 -7.27104675e-03 -1.64662339e-02
 -8.97176042e-02 -3.57704498e-02 -8.83165002e-03 -1.07549746e-02
  1.25099597e-02 -5.96218929e-02 -5.44408560e-02 -3.52668352e-02
 -7.68604130e-02  4.23438735e-02 -4.16345224e-02 -6.90007433e-02
  2.26748325e-02  2.54301839e-02 -8.09754431e-02  2.43239794e-02
  7.86312595e-02  1.84833780e-02  1.08493213e-02 -4.64129411e-02
 -1.10290259e-01 -6.91219345e-02 -8.26006085e-02  8.85387957e-02
  5.45853935e-02  3.15184891e-03 -2.82067806e-02  5.31568341e-02
  3.25025432e-02 -7.25305751e-02  1.47844227e-02 -2.62511894e-03
 -7.01897964e-02  4.82845977e-02  1.42504117e-02  8.28930177e-03
  1.66053623e-02  3.81776132e-02 -3.53184007e-02  2.71549672e-02
 -1.90307498e-02  6.52991235e-02 -2.72371166e-04 -2.06697471e-02
 -5.50780743e-02 -5.32911681e-02  3.92511860e-02  1.78272165e-02
  6.61730990e-02  4.86668199e-02 -3.19321686e-03 -7.11495057e-02
 -3.71031873e-02  1.00502349e-01 -8.23945403e-02  8.04896355e-02
  1.71221304e-03 -1.83992460e-01 -1.66723151e-02 -1.69434622e-02
 -2.24867854e-02 -1.71718337e-02  2.27608625e-02 -5.56295328e-02
 -5.51749319e-02 -4.00115317e-03  2.56852936e-02  1.90157425e-02
  4.50189598e-02 -6.62937760e-02 -4.37983684e-02 -2.81101968e-02
  7.72700608e-02  1.38412341e-02  5.67251816e-02  2.50283256e-02
  1.28587463e-03 -1.85612328e-02  5.00082821e-02 -3.46653610e-02
  5.71519732e-02 -2.21766215e-02 -1.93323102e-02 -2.75285542e-02
  7.68124163e-02 -7.09496439e-02 -1.56743489e-02 -5.67429233e-33
  7.19574243e-02  3.65115479e-02  3.06076440e-03  9.03434604e-02
  1.80814415e-02  2.34081224e-02  6.71167821e-02  9.41870734e-03
  3.10891103e-02 -6.35216758e-02  5.71906054e-03 -5.41443117e-02
  2.35615717e-03  6.51084557e-02 -1.61613580e-02  4.95714461e-03
```

```
 -4.75244364e-03  6.51564747e-02 -1.81264132e-02 -3.79895954e-03
 -1.39650032e-02  4.02317792e-02  6.20299801e-02  4.54756357e-02
  1.66739926e-01  4.20602039e-02  2.50018425e-02 -2.31871214e-02
  1.91103797e-02  3.80754545e-02 -7.85840442e-04 -6.99788844e-03
 -9.93106794e-03  8.62581376e-03 -4.89946418e-02  4.96172905e-02
  2.97412816e-02 -6.42179474e-02  5.97528368e-02  1.10771209e-02
  5.40427566e-02 -9.04934108e-02  1.14269694e-02  2.36828532e-02
 -3.84702832e-02 -8.04097280e-02 -3.09143141e-02 -3.33753042e-02
 -7.71655068e-02  6.86534643e-02 -4.40332070e-02  1.25444531e-02
 -8.11590254e-03 -6.15915656e-03 -6.21482637e-03 -2.16211975e-02
  3.92784402e-02 -4.26357277e-02 -6.31945580e-02 -3.54904011e-02
  1.28261102e-02  5.96963726e-02  1.20180380e-02 -3.67399566e-02
 -1.87519211e-02 -3.93126644e-02 -6.34405464e-02  3.20547190e-03
  7.02882409e-02  5.78786470e-02 -1.27664879e-01 -4.07377668e-02
 -1.85519923e-02 -8.26592650e-03 -2.46807542e-02 -1.09196855e-02
 -1.56567525e-02 -3.32213603e-02  1.19540796e-01  4.43487043e-05
 -5.88751659e-02 -4.14109491e-02 -1.10954925e-01  2.82492815e-03
  4.72160876e-02  2.45506899e-03  2.98352428e-02 -1.20533155e-02
 -4.53316187e-03 -3.00619230e-02  2.85341945e-02  2.22048853e-02
 -7.63275288e-03  1.45907745e-01  1.11934971e-02  1.89330355e-33
  8.86925384e-02  3.67788002e-02  9.51909050e-02  7.49003515e-02
```

```
# Save the FAISS index to disk
vectorstore.save_local("faiss_index")
```

```
# Step6: Initialize GPT-4 Turbo via LangChain
# For the Chat/Language Model part of your pipeline, use ChatGroq:

llm = ChatGroq(api_key=groq_api_key, model="llama-3.1-8b-instant")
```

```
# Step7: Create a RetrievalQA chain (retriever + LLM)

"""Converts your vectorstore (FAISS in this case) into a retriever object.
A retriever knows how to fetch relevant chunks from the vector store using a query.
search_kwargs={"k": 3} - return the top 3 most similar chunks (based on vector similarity) for any question."""
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})

rag_chain = RetrievalQA.from_chain_type(
    llm=llm, # Use the initialized LLM (GPT-4 Turbo)
    retriever=retriever, # Use the retriever to fetch relevant chunks
    return_source_documents=True # Return the source documents along with the answer
)
```

```
from IPython.display import Markdown, display
```

```
def clear(text):
  return display(Markdown(text))
```

```
# Step 8: Function to ask questions based on the PDF content
def ask_pdf_agent(query: str):
    print(f"\nUser Query: {query}")
    result = rag_chain.invoke({"query": query})
    # print("=="*30)
    # print(f"Entire Result: {result}")
    # print("--"*30)
    # print(len(result["source_documents"]))
    # print("=="*30)
    print()
    print("\nAnswer:")
    clear(result["result"])
    print("=="*80)
    print("=="*80)

    print("\nRetrieved Passages:")
    for doc in result["source_documents"]:
        clear(f"==> {doc.page_content.strip()[:200]}...")
```

```
# Step 9: Run a sample query
ask_pdf_agent("What is  THE VANILLA LSTM NETWORK MECHANISM IN DETAIL?")
```

```
User Query: What is  THE VANILLA LSTM NETWORK MECHANISM IN DETAIL?
```

```
Answer:
```

The Vanilla LSTM (Long Short-Term Memory) network is a type of Recurrent Neural Network (RNN) designed to handle the vanishing gradient problem in traditional RNNs. It uses a memory cell to store information for a long period, allowing the network to learn long-term dependencies.

The Vanilla LSTM network cell's schematic diagram is composed of several major stages, which are marked by dashed rectangles. The following are the key components of the Vanilla LSTM network:

1. **Input Gate**: This stage controls the amount of new information that is added to the memory cell. It multiplies the input with a learned weight (gate weight) and adds a bias term. The output of this stage is a scalar value between 0 and 1, indicating the amount of new information to be added to the memory cell.
2. **Forget Gate**: This stage determines how much of the current memory cell's information should be forgotten. It multiplies the previous memory cell's output with a learned weight (gate weight) and adds a bias term. The output of this stage is a scalar value between 0 and 1, indicating how much of the current memory cell's information should be forgotten.
3. **Cell State**: This stage updates the memory cell's state based on the outputs of the input gate and forget gate. The cell state represents the long-term memory of the network.
4. **Output Gate**: This stage controls the amount of information that is output from the memory cell. It multiplies the cell state with a learned weight (gate weight) and adds a bias term. The output of this stage is the final output of the network.

The key equations for the Vanilla LSTM network are:

- **forget gate output**: ft = σ(Wf * [ht-1, xt] + bf)
- **input gate output**: it = σ(Wi * [ht-1, xt] + bi)
- **cell state update**: ct = ft * ct-1 + it * tanh(Wc * [ht-1, xt] + bc)
- **output gate output**: ot = σ(Wo * [ht-1, xt] + bo)
- **output**: ht = ot * tanh(ct)

where:

- Wf, Wi, Wc, Wo: weight matrices for the forget, input, cell, and output gates, respectively.
- bf, bi, bc, bo: bias terms for the forget, input, cell, and output gates, respectively.
- σ: sigmoid activation function.
- tanh: hyperbolic tangent activation function.
- ht-1: previous hidden state.
- xt: current input.
- ct-1: previous cell state.
- ct: updated cell state.
- ht: final hidden state.

The Vanilla LSTM network's architecture is designed to handle long-term dependencies by using a memory cell that can store information for a long period. The input gate, forget gate, cell state, and output gate work together to control the flow of information through the network, allowing it to learn complex patterns and relationships in data.

==========
==========

```
Retrieved Passages:
```

==> performed by the components of the Vanilla LSTM cell, its schematic diagram is redrawn in Figure 9, with the major stages comprising the cell's architecture marked by dashed rectangles annotated by th...

==> VII. E XTENSIONS TO THE VANILLA LSTM N ETWORK Since its invention, many variants and extensions of the original LSTM network model have been researched and utilized in practice. In this section, we wi...

==> explanation of all aspects of the Vanilla LSTM cell. Even though this section is intended to be self-contained, familiarity with the material covered in the preceding sections will be beneficial. The

```
ask_pdf_agent("What were the RNN TRAINING DIFFICULTIES")
```

```
User Query: What were the RNN TRAINING DIFFICULTIES
```

```
Answer:
```

The text doesn't explicitly state the RNN training difficulties. It mentions that a detailed treatment of the difficulties encountered in training RNNs is presented in [44], but it doesn't elaborate on what those difficulties are.

==========
==========

```
Retrieved Passages:
```

==> occupy the main diagonal. 14A detailed treatment of the difficulties encountered in training RNNs is presented in [44]. The problem is defined using Equation 80, which leads to the formulas for the gr...

==> 2An article co-authored by one of the LSTM inventors provides a self-contained summary of the embodiment of an RNN, though not at an introductory level [17]. 2...

==> IV. RNN T RAINING DIFFICULTIES Proposition 1 establishes that Equations 73 – 77 together with Equation 42 specify the truncated unrolled RNN system that realizes the standard RNN system, given by Equa...

```
ask_pdf_agent("what is chain-of-Thought Prompting and what are its advantages?")
```

```
User Query: what is chain-of-Thought Prompting and what are its advantages?


Answer:
```

To answer this question, I'll follow a step-by-step thought process to arrive at the answer.

1. **Understanding the concept of Chain-of-Thought Prompting**: I need to understand what Chain-of-Thought Prompting (COT) is. Based on the provided text, I can infer that COT involves providing a language model with a series of thought steps or intermediate reasoning to help it solve a problem.
2. **Decomposing the problem into smaller steps**: The text mentions that COT allows models to decompose multi-step problems into smaller, manageable steps. This suggests that COT is a strategy to facilitate step-by-step reasoning in language models.
3. **Analyzing the advantages of COT**: The text highlights that COT has several attractive properties, including allowing models to decompose multi-step problems, providing a more transparent and interpretable reasoning process, and potentially improving performance on challenging tasks.
4. **Identifying the key conditions for COT to be effective**: According to the text, COT is more helpful for tasks that meet three conditions: (1) the task is challenging and requires complex reasoning, (2) the task can be decomposed into smaller steps, and (3) the task is a text-to-text task.

Based on these steps, here's the answer:

Chain-of-Thought Prompting (COT) is a strategy that involves providing a language model with a series of thought steps or intermediate reasoning to help it solve a problem. This approach allows models to **decompose multi-step problems into smaller, manageable steps**, making it more effective for **challenging tasks that require complex reasoning**, such as those that can be **decomposed into smaller steps** and are **text-to-text tasks**. The advantages of COT include providing a more **transparent and interpretable reasoning process**, potentially **improving performance**, and allowing models to **learn from intermediate reasoning**.

References: Narang, S., et al. (2020) Wiegreffe, A., et al. (2022) Lampinen, A., et al. (2022)

```
==========================================================================================================================
==========================================================================================================================

Retrieved Passages:
```

==> mimics a step-by-step thought process for arriving at the answer (and also, solutions/explanations typically come after the final answer (Narang et al., 2020; Wiegreffe et al., 2022; Lampinen et al., 2...

==> with chains of thought for prompting—Figure 1 (right) shows one chain of thought exemplar, and the full set of exemplars is given in Appendix Table 20. (These particular exemplars did not undergo prom...

==> A.3 Will chain-of-thought prompting improve performance for my task of interest? While chain-of-thought prompting is in principle applicable for any text-to-text task, it is more helpful for some task...

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.